

06 | Pressable：如何实现一个体验好的点按组件？

2022-4-8 蒋宏伟



你好，我是蒋宏伟。

点按组件的设计与我们的用户体验息息相关。有人会因为机械键盘的敲击感好，不买百来块的薄膜键盘，而花上贵十倍的价格去买 HHKB、Filco，也有人会因为某个应用的点按体验不好，而转投竞品应用。

如果你仔细观察过世界上那些流行的、口碑很好 App，比如微信，你会看到它们在点按组件的体验的细节上都做得特别好。

比如，微信的点按组件都是有交互反馈的，无论是背景颜色的加深，还是那些舒服的震动，又或者是动画。又比如，微信顶部右上角的加号按钮是很容易点击的，它的点击区域是比显示图标大上那么一丢丢，而且点到后，即使把手指挪开图标的位置再松开也是能触发点击的。

所有的这些设计都是“懂”用户的。担心你因为网络卡、机器卡不知道有没有自己点中，在你点完后给你视觉或触觉上的反馈；担心你走路的时候想点点不到，把事件的“可触发区

域”、“可保留区域”设置得比视觉上的“可见区域”大上那么一些。

作为直接和用户打交道的工程师，**我们**也得“懂”用户，也得去优化我们负责的 App、页面的体验，**还得在技术上搞懂点按组件使用方法和背后的原理，把这种最常用的人机交互体验给做到及格，做到优秀。**

所以，今天这节课，我会以三个问题为脉络进行讲解：

点按组件是要简单易用还是要功能丰富，如何取舍？

点按组件是如何知道它是被点击了，还是被长按了？

点按组件为什么还要支持用户中途取消点击？

通过这三个问题，你不仅能明白如何在 React Native 中实现一个体验好的点按组件，同时也能借助它背后的设计原理，更“懂”用户，提升产品的用户体验。

要简单易用还是功能丰富？

首先，点按组件是设计给你我这样的开发者来使用的，它功能越简单开发者用起来就越简单，它功能越复杂就能满足更多的需求场景。那是让开发者简单易用好，还是用丰富的功能去满足用户，有没有两全其美之计？

实际上，React Native 的点按组件经历了三个版本的迭代，才找到了两全其美的答案。等你了解了这个三个版本的迭代思路后，你就能很好明白优秀通用组件应该如何设计，才能同时在用户体验 UX 和开发者体验 DX 上找到平衡。

我先给你从第一代点按组件开始讲起。

第一代 Touchable 组件

第一代点按组件想要解决的核心问题是，**提过多种反馈风格。**

一个体验好的点按组件，需要在用户点按后进行实时地反馈，通过视觉变化等形式，告诉用户点到了什么，现在的点击状态又是什么。

但不同的原生平台，有不同的风格，反馈样式也不同。Android 按钮点击后会有涟漪，iOS 按钮点击后会降低透明度或者加深背景色。React Native 是跨平台的，那它应该如何支持多种

平台的多种反馈风格呢？

第一代 Touchable 点按组件的设计思路是，提供多种原生平台的反馈风格给开发者自己选择。框架提供了 1 个基类和 4 个扩展类，它们分别是：

TouchableWithoutFeedback：用于响应用户的点按操作，但不给出任何点按反馈效果。反馈效果由 4 个扩展类实现；

TouchableNativeFeedback：给出当前原生平台的点按反馈效果，在 Android 中是涟漪（ripple）效果，就是从点击处散开水波纹的效果；

TouchableOpacity：短暂地改变组件的透明度；

TouchableHighlight：短暂地加深组件的背景色；

TouchableBounce：有 bounce 回弹动画的响应效果。

Touchable 点按组件提供了 5 个类，选择起来也很麻烦。有经验的开发者可能知道如何进行选择，但新手却要花上很长时间，去了解不同组件之间的区别。所以说，Touchable 点按组件在提供多样性的功能支持的同时，也带来了额外的学习成本。

为了降低学习成本，React Native 团队又开发了第二代点按组件，Button。

第二代 Button 组件

第二代 Button 组件的实质是**对 Touchable 组件的封装**。在 Android 上是 TouchableNativeFeedback 组件，在 iOS 上是 TouchableOpacity 组件。

Button 组件的设计思想就是，别让开发者纠结选啥组件了，框架已经选好了，点按反馈的风格就和原生平台的自身风格保持统一就好了。

但我的经验告诉我，要让大多数开发者都选择同一个默认的 UI 样式真是太难了，萝卜白菜各有所爱。另外，用户的审美也在慢慢地变化，涟漪风格也好，降低透明风格也好，背景高亮风格也好，或许几年后就不会再流行了。甚至连 Button 这个概念本身，都在慢慢地变化，现在的 App 中几乎只要是个图片或者文字都能点按，不再局限于只有四四方方的色块才能点按了。

Button 组件虽然降低了开发者选择成本，但是想在 UI 风格上让大家选择都原生平台自身的风格，这太难了。因此，React Native 团队又开发了第三代点按组件 Pressable。

第三代 Pressable 组件

第三代 Pressable 点按组件，不再是 Touchable 组件的封装，而是一个**全新重构的点按组件**，它的反馈效果可由开发者自行配置。

但是，点按组件通常是有点击和未点击两种状态的，这两种状态对应着两种点按样式，一种样式是未点击时的基础样式，一种是点按后的反馈样式。这两种样式怎么写？又该怎么切换？

Pressable 组件的 API 设计得很是巧妙，扩展起来非常方便。Pressable 的样式 style 属性同时支持固定样式，和函数返回的“动态样式”：

```
type PressableStyle = ViewStyle | (({ pressed: boolean }) => ViewStyle)
```

其一，固定样式，也就是`type PressableStyle = ViewStyle`的意思是，Pressable 组件的支持样式类型和 View 组件的支持样式类型是一样的，具体 `ViewStyle` 都包括那些“通用”样式和“私有”样式，我们在 [《Style》](#) 中已经学过了，相信你能很快回想起来。

其二，动态样式，也就是`type PressableStyle = (({ pressed: boolean }) => ViewStyle)` 的意思是，在用户没有点击时 `pressed` 值为 `false`，在用户点击时 `pressed` 值为 `true`，你可以根据两种点按状态，为按钮定制不同的样式。

具体怎么实现呢？我们先来看固定样式。固定样式，顾名思义，就是按钮组件的样式是“固定”的，比如你可以看下这段代码：

```
// 固定的基础样式
const baseStyle = { width: 50, height: 50, backgroundColor: 'red' }

<Pressable
  onPress={handlePress}
  style={baseStyle} >
  <Text>按钮</Text>
</Pressable>
```

这段示例代码就是一个最简单的固定样式按钮的代码片段。我们在 `Pressable` 元素中嵌套了一个文字是“按钮”的 `Text` 元素，并给 `Pressable` 元素添加了一个固定的基础样式，宽高各位 50 像素，且背景颜色为红色。

那如果我们需要实现动态样式，应该怎么实现呢？比如，你想在所有平台都实现降低透明度的点击反馈，那你可以定义一个基础样式 `baseStyle`，然后通过点按状态 `pressed`，管理透明度 `opacity` 的切换。具体的代码示例如下：

```
// 固定的基础样式
const baseStyle = { width: 50, height: 50, backgroundColor: 'red'}

<Pressable
  onPress={handlePress}
  style={({ pressed }) => [ /* 动态样式 */
    baseStyle,
    { opacity: pressed ? 0.5 : 1 }
  ]} >
  <Text>按钮</Text>
</Pressable>
```

这段示例代码用的就是 `Pressable` 的动态样式。首次渲染时，React Native 会先调用一次 `Pressable` 的 `style` 属性的回调函数，这时点按状态 `pressed` 是 `false`，透明度为 1。在你触碰到“按钮”时，就会触发点击事件 `onPress`，与此同时，React Native 会再调用一次 `style` 属性的回调函数，此时点按状态 `pressed` 是 `true`，透明度为 0.5。在你松开“按钮”后，透明度会重新变为 1。

你可以看到，使用动态样式来实现降低透明度的点击反馈是非常方便的。除了改变透明度，你还可以选择改变背景色，改变按钮的宽高，甚至还可以把“按钮”的文字改了。你看，动态样式是不是非常灵活？

除了这两点，你可能还会问，如果我想实现 Android 平台特有的涟漪效果，`Pressable` 组件也能实现吗？可以，你可以使用 `android_ripple` 和 `android_disableSound` 属性进行配置。

`android_ripple`：用于配置 Android 特有的涟漪效果 [RippleConfig](#)；

`android_disableSound`：禁用 Android 系统的点击音效，默认 `false` 不禁用。

其实，目前这三代点按组件是同时存在于 React Native 的官方组件库中的，那开发时我们该怎么选呢？我认为：

第一代点按组件 Touchable，功能丰富但学习成本太高；

第二代点按组件 Button，简单易用但带了默认样式和反馈效果，通用性太差；

第三代点按组件 Pressable，同时满足了简单易用和复杂效果可扩展的特性。

因此，在实现自定义的业务按钮组件时，我更加推荐你使用第三代点按组件 Pressable。而且，Pressable 组件的动态 style 的设计思路，也是非常值得我们学习的。

如何知道是点击，还是长按？

我们再来看第二个问题：点按组件 Pressable 是如何知道它是被点击了，还是被长按了？

整个点按事件的响应过程是硬件和软件相互配合的过程。Pressable 组件响应的整体流程，是从触摸屏识别物理手势开始，到系统和框架 Native 部分把物理手势转换为 JavaScript 手势事件，再到框架 JavaScript 部分确定响应手势的组件，最后到 Pressable 组件确定是点击还是长按。

你看，一个 App 要识别是点击还是长按，并没有那么容易吧？庆幸的是，这些复杂的识别工作都由手机硬件、操作系统、React Native 框架帮我们实现了。作为开发者，大部分时候我们只需要知道怎么使用 and 了解基本原理就可以了。今天我们把焦点放在最后一步，Pressable 组件是怎么确定用户是点击还是长按的。

我们知道，开始响应事件和结束响应事件是两个最基础的手势事件，在 Android、iOS 或者 Web 中都有类似的事件。在 React Native 中它们是：

onResponderGrant：开始响应事件，用户手指接触屏幕，且该手势被当前组件锁定后触发；

onResponderRelease：结束响应事件，用户手指离开屏幕时触发。

基于开始响应事件 onResponderGrant 和结束响应事件 onResponderRelease，Pressable 组件可以很容易地封装出开始点按事件 onPressIn 和结束点按事件 onPressOut。

你可以在 Pressable 组件中，使用 onPressIn 来响应开始点按事件，使用 onPressOut 来响应结束点按事件。示例代码如下：

```
<Pressable
  onPressIn={handlePressIn}
  onPressOut={handlePressOut}
>
  <Text>按钮</Text>
</Pressable>
```

当你触碰到“按钮”开始点按时，React Native 框架就会帮你调用 handlePressIn 处理函数，当你手指离开“按钮”结束点按时，就会调用 handlePressOut 处理函数。

基于开始点按事件 onPressIn 和结束点按事件 onPressOut，我们自己是否可以封装出“自定义”的点击事件 onPress 和长按事件 onLongPress 呢？你可以短暂的按一下暂停键，思考一下如果要你来实现你会怎么做，然后再去看 React Native 框架提供的答案。

这个方案也很简单，你只需要判断 onPressIn 事件和 onPressOut 事件之间触发间隔耗时就可以了：

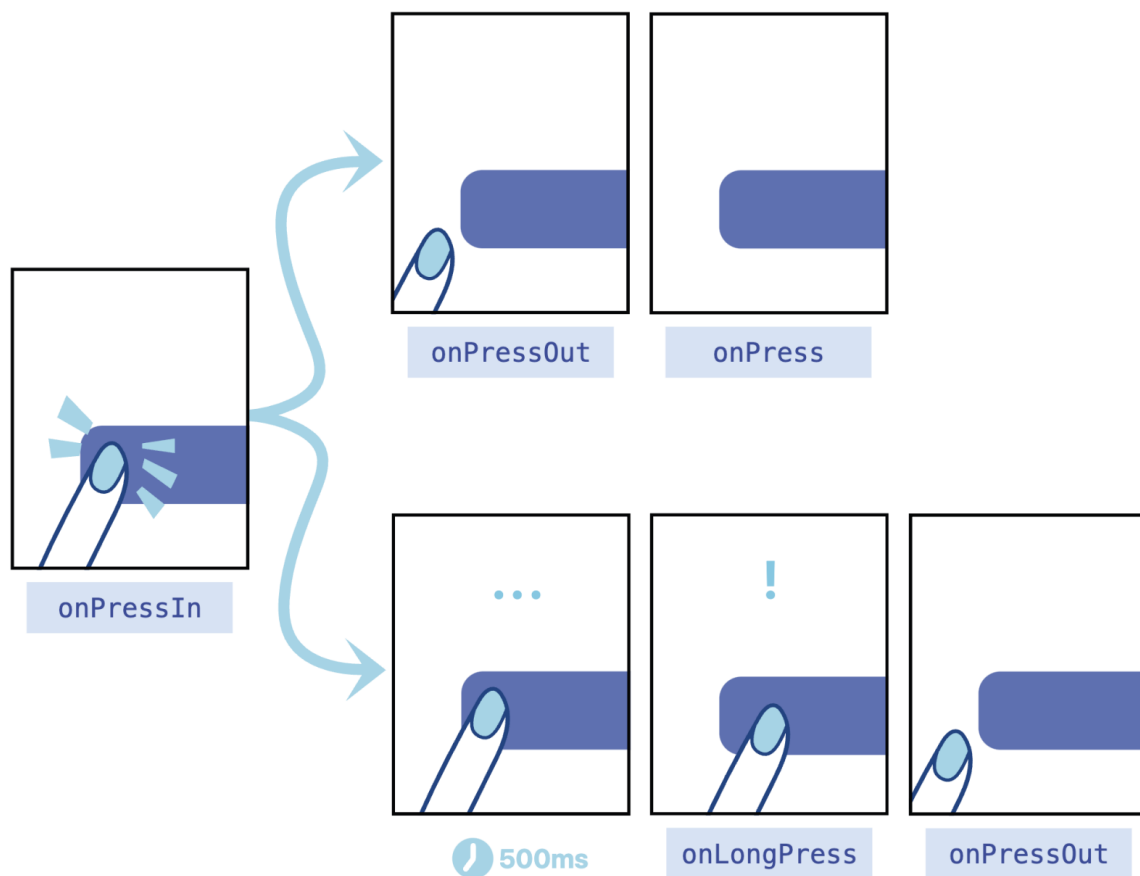
如果间隔耗时 $< 500\text{ms}$ 属于点击。用户的点按动作会先触发 onPressIn，再触发 onPressOut，在 onPressOut 事件中可以触发我们“自定义的”点击事件 onPress；

如果间隔耗时 $\geq 500\text{ms}$ 属于长按。用户的点按动作会先触发 onPressIn，这个时候你可以埋下一个定时器，并在第 500ms 时通过定时器触发我们“自定义的”onLongPress，最后在用户松手的时候触发 onPressOut。

实际上，React Native 框架就是这么设计的。

在你同时监听了 onPress 和 onLongPress 两个事件时，如果点按耗时小于 500ms，在你松手时触发的是点击事件 onPress；如果点按耗时大于 500ms，大致会在第 500ms 先触发长按事件 onLongPress，那这时即使你再松手也不会触发 onPress 事件了。也就是说，**点击事件 onPress 和长按事件 onLongPress 是互斥的，触发了一个就不会再触发另一个了。**

关于 Pressable 组件的 4 个响应事件，onPressIn、onPressOut、onPress 和 onLongPress 的触发方式，我放了一张官方提供的示意图，相信你看后会有更深的理解：



为什么支持中途取消？

现在，你对 Pressable 组件的点按事件的工作原理已经有所了解了。讲到这里，我们开头提出的三个问题，只剩最后一个：点按组件为什么还要支持用户中途取消点击？

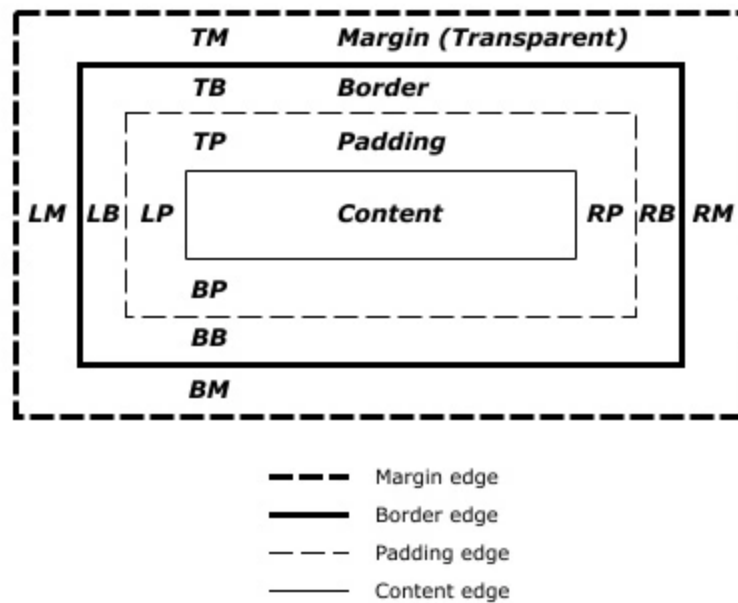
要讲清楚这个问题，我们需要深入到事件区域模型，也就是点按操作手势的可用范围的概念下进行讲解。

点按操作手势的可用范围包括盒模型区域、可触发区域 HitRect 和可保留区域 PressRect，接下来我们一个个讲解。

盒模型区域

还记得吗？在我们介绍 [《Style》](#) 的布局属性时，我们有提到过宽度 width、高度 height，这两个属性就决定了盒模型（Box Model）中的内容 content 大小。除此之外，盒模型中还有内边距 padding、边框 border、外边距 margin。

这些内容、边框、边距之间关系是什么呢？其实，React Native 中的盒模型概念来自于 Web 领域的 W3C 规范，我把规范中的盒模型示意图放在了下面：



你可以看到，最里面的是内容 Content，然后再是 Padding 和 Border，最外面的才是 Margin。请你注意了，Content、Padding、Border 默认是不透明度的，但 Margin 是天生透明的，并且不可以设置透明度、设置颜色。

你猜，点按事件的默认触发区域是盒模型中的哪几部分？答案就是，盒模型中的默认不透明的部分。这些用户看得见的部分，包括 content、padding 和 border 部分。可以看得见才可以点击，这样的设计是非常合理的。

我给你贴出了点按事件的默认触发区域的测试代码，你也可以自己点一点、试一试，体会一下：

```
<Pressable style={{
  margin: 10,
  borderWidth: 10,
  borderColor: 'red',
  padding: 10,
  width: 100,
  height: 100,
  backgroundColor: 'orange',
}}>
  <Text>点我</Text>
</Pressable>
```

在上面的示例代码中，我们特意给了 100 像素的宽高，这是很容易点中的，但在日常我们使用 App 时，并不会有这么大按钮。你也许遇到过类似的情景，单手把持手机的时候左上角的返回键老点不中，勾选用户同意事项的时候老勾不中，等等。人的手指并不是什么精密仪器，不能保证任何情况下都能正确地点按到指定区域。那这种情况该怎么处理呢？

我们可以直接修改宽高、边框、内边距的值，通过扩大盒模型的范围，提高点中的成功率。但是，修改盒模型成本较高，它可能会导致原有 UI 布局发生变化。

更好的方案是，不修改影响布局的盒模型，直接修改可触发区域的范围，提高点中的成功率。

可触发区域 HitRect

Pressable 组件有一个可触发区域 HitRect，默认情况下，可触发区域 HitRect 就是盒模型中的不透明的可见区域。你可以通过修改 hitSlop 的值，直接扩大可触发区域。

HitSlop 类型的定义如下：

```
type Rect = {
  top?: number;
  bottom?: number;
  left?: number;
  right?: number;
}

type HitSlop = Rect | number
```

HitSlop 接收两种类型的参数，一种是 number 类型，以原有盒模型中的 border 为边界，将可触发区域向外扩大一段距离。另一种是 Rect 类型，你可以更加精准地定义，要扩大的上下左右的距离。

在老点不中、老勾不中的场景中，你可以在不改变布局的前提下，设置 Pressable 组件的可触发区域 HitSlop，让可点击区域多个 10 像素、20 像素，让用户的更容易点中。

可保留区域 PressRect

前面我讲到，用户的手势可能会有一定误差。不仅如此，用户的行为本身就很复杂，用户的意愿也可能会在很短的时间内发生改变的。其实，这里也是在回答开头我们提出的最后一个问题，用户行为的复杂性，就导致了我们在设计点按组件需要有更多的思考。

比如，用户已经点到购买按钮了，突然犹豫，又不想买了，于是将手指从按钮区域移开了。这时你得让用户能够反悔，能够取消即将触发的点击操作。

这里我们就要引入一个新的概念：可保留区域 `PressRect`。点按事件可保留区域的偏移量（`Press Retention Offset`）默认是 0，也就是说默认情况下可见区域就是可保留区域。你可以通过设置 `pressRetentionOffset` 属性，来扩大可保留区域 `PressRect`。
`pressRetentionOffset` 属性的类型如下：

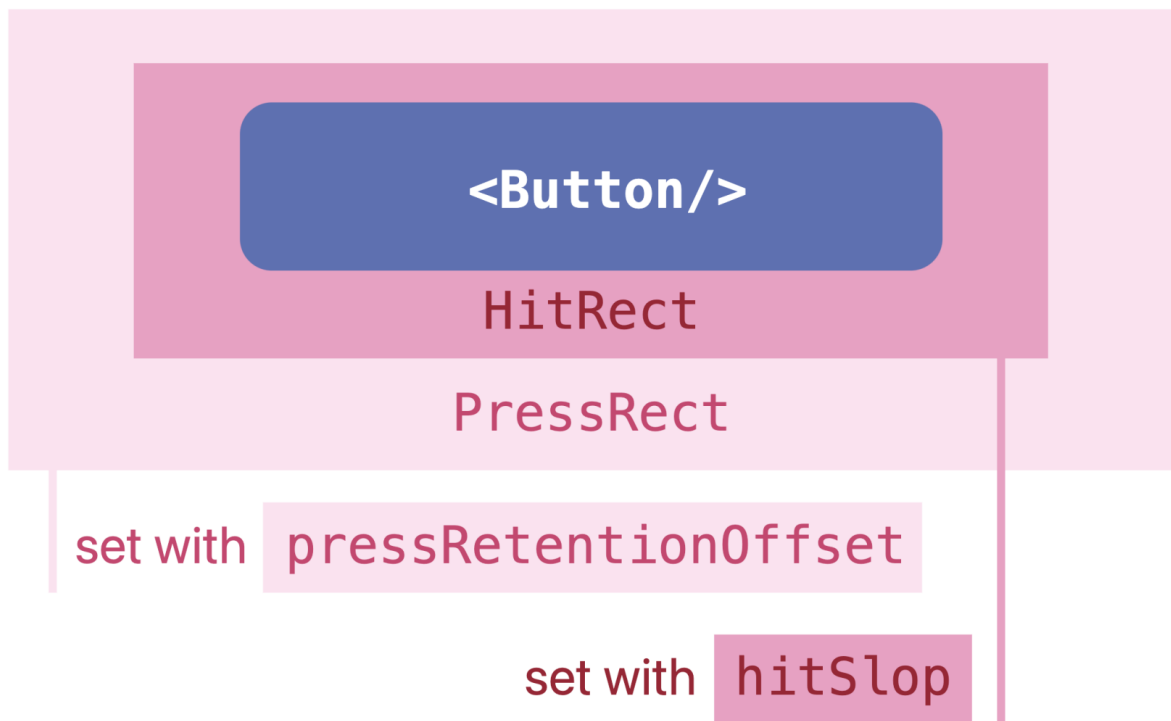
```
type PressRetentionOffset = Rect | number
```

你可以看到，`pressRetentionOffset` 和 `HitSlop` 一样，接收两种类型的参数，一种是 `number` 类型，另一种是 `Rect` 类型。`Rect` 类型设置后，会以原有可触发区域为基准，将可保留区域向外扩大一段距离。

在你后悔点下购买按钮的情况下，你可以把已经按下的手指从可保留区域挪开，然后再松手，这就不会再继续触发点击事件了。

当然，还有更复杂的情况，你已经点到购买按钮了，突然犹豫，开始进行心理博弈，想点又不想点。手指从按钮上挪开了，又挪了进去，然后又挪开了，如此反复。这时还要不要触发点击事件呢？要不要触发，其实是根据你手指松开的位置来判断的，如果你松手的位置在可保留区域内那就要触发，如果不是那就不触发。

我将盒模型区域的可见区域、可触发区域 `HitRect` 和可保留区域 `PressRect` 的关系画了一张图，你也可以打开文稿看看，加深一下理解：



课程小结

以上就是我们这一讲的全部内容，现在我来给你总结一下今天这一讲的要点。如何实现一个体验好的点按组件呢？我建议你记住下面这三点：

首先，一个好的点按组件应该先让开发者用起来很方便。React Native 的点按组件经历了三次迭代，每次迭代都在开发者体验（DX）上有所进步，我更推荐你使用第三代点按组件 `Pressable`。

其次，一个好的点按组件应该要满足各种用户、各种场景的可扩展性。`Pressable` 组件支持四种基础点按事件，`onPressIn`、`onPressOut`、`onPress`、`onLongPress`。其中，点击事件 `onPress` 和长按事件 `onLongPress` 是互斥的，触发了一个就不会再触发另一个了。

最后，一个优秀的工程师应该要“懂”用户，要把自己负责的 App、页面的用户体验（UX）提上去。任何的物理按钮都是有点击反馈的，我们的虚拟按钮也得有，这是最基本体验要求。然后，要让用户想点能点得到，要理解盒模型区域、可触发区域 `HitRect`、可保留区域 `PressRect` 的区别，并且进行合理设置。

补充材料

官方文档：

TouchableHighlight: <https://reactnative.dev/docs/next/touchablehighlight>

Button: <https://reactnative.dev/docs/next/button>

Pressable: <https://reactnative.dev/docs/next/pressable>

源码阅读:

要读懂点按组件 Pressable 的核心设计原理，首先要读懂 Pressable 设计者的设计思想，[它放在了 Pressability.js 文件中。](#)

点按组件 Pressable 的 4 种基础响应事件是基于[手势系统](#)实现的，其中 [onPress](#) 和 [onLongPress](#) 是互斥的。

作业

1. 请你模仿实现一下微信顶部右上角的加号按钮。
2. 在较老版本的手机浏览器中，点击事件存在 350ms 延迟；在微信聊天框中，点击对方的微信头像比点击右上角三个点的更多按钮，打开页面的速度慢一些；双击事件是常见的点按事件之一，Pressable 组件却没有提供；这三个现象涉及到了 Web、Android、iOS 和 React Native 这四个技术领域，但这三个现象其实都指向同一个答案。欢迎你把你的答案分享给大家。

我是蒋宏伟，咱们下节课见。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 05 | Image: 选择适合你的图片加载方式

下一篇 07 | TextInput: 如何实现一个体验好的输入框?